Towards Contact-Aided Motion Planning for Tendon-Driven Continuum Robots

Priyanka Rao, *Graduate Student Member, IEEE*, Oren Salzman, *Member, IEEE*, and Jessica Burgner-Kahrs, *Senior Member, IEEE*

***Abstract*—Tendon-driven continuum robots (TDCRs), with their flexible backbones, offer the advantage of being used for navigating complex, cluttered environments. However, to do so, they typically require multiple segments, often leadingto complex actuation and control challenges. To this end, we propose a novel approach to navigate cluttered spaces effectively for a single-segment long TDCR which is the simplest topology from a mechanical point of view. Our key insight is that by leveraging contact with the environment we can achieve multiple curvatures without mechanical alterations to the robot. Specifically, we propose a search-based motion planner for a single-segment TDCR. This planner, guided by a specially designed heuristic, discretizes the configuration space and employs a best-first search. The heuristic, crucial for efficient navigation, provides an effective cost-to-go estimation while respecting the kinematic constraints of the TDCR and environmental interactions. We empirically demonstrate the efficiency of our planner—testing over 525 queries in environments with both convex and non-convex obstacles, our planner is demonstrated to have a success rate of about 80% while baselines were not able to obtain a success rate higher than 30%. The difference is attributed to our novel heuristic which is shown to significantly reduce the required search space.**

***Index Terms*—Modeling, control, and learning for soft robots, motion and path planning, soft robot applications.**

I. Introduction

TENDON-DRIVEN continuum robots (TDCRs), bend their flexible backbones via tendon actuation, making them popular for medical and industrial applications for accessing areas with constricted entry-points. As navigating cluttered areas requires more intricate shapes, several segments are typically used, as each segment is capable of bending only in a C-shape. A multi-segment design comes at the price of complex actuation unit design [1] and tendon control. While methods like variable tendon routing [2], layer jamming [3], and locking mechanisms [4] vary curvatures, they demand mechanical modifications to the robot design. In this work we argue that in cluttered environments, we can use a simple single-segment long TDCR that exploits contacts with the environment (*contact-aided navigation (CAN)*) to achieve variations in curvature.

TDCRs can be used to deviate from the common obstacle-avoidance paradigm, as they can interact with the environment due to their compliance. For example, they have been exploited for environmental sensing by actively tapping the surroundings [5]. They can be used for CAN as the environmental interactions deform the robot due to resulting external constraints on the robot shape, providing passive degrees of freedom (DOF) that can be leveraged to improve the workspace that the robot can access [6]. However, in scenarios cluttered with obstacles where the robot's end-effector must reach a target pose in order to carry out various tasks, a vital component is a motion planner capable of making combinatorial decisions about the necessary contacts to reach the desired pose. This motion planner, which is the focus of this letter, is responsible for identifying the sequence of actuation inputs required to navigate the robot.

Consider an environment with obstacles, where the robot needs to reach a target pose (see Fig. 1(a)). A single-segment robot has limited workspace and cannot achieve the illustrated

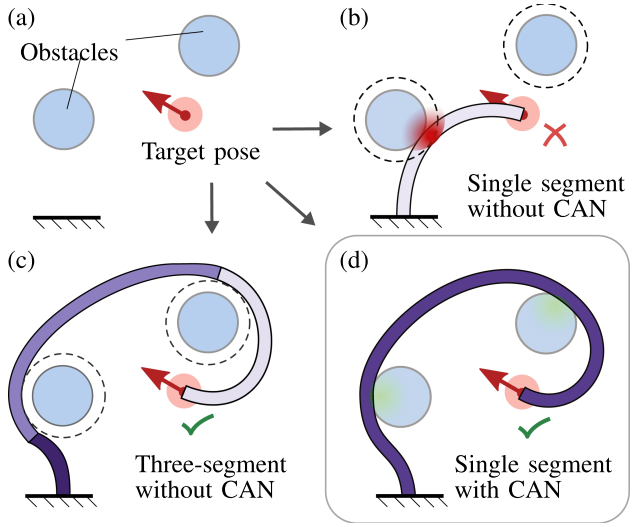


Fig. 1. (a) Problem statement: the end-effector of a TDCR must reach a given target pose in a cluttered environment, (b) Single-segment TCDR's unsuccessful solution (without CAN) which intersects an obstacle, (c) Three-segment TDCR's successful solution, (d) Single-segment's successful solution (with CAN) obtained by leveraging the obstacles to vary its curvature and reach the target pose.

Manuscript received 2 December 2023; accepted 21 March 2024. Date of publication 29 March 2024; date of current version 9 April 2024. This letter was recommended for publication by Associate Editor F. Renda and Editor C. Laschi upon evaluation of the reviewers' comments. This work was supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) under Grant RGPIN-2019-04846, in part by the Israeli Ministry of Science & Technology under Grant 3-16079 and Grant 3-17385, and in part by the United States-Israel Binational Science Foundation (BSF) under Grant 2019703. *(Corresponding author: Priyanka Rao.)*

Priyanka Rao and Jessica Burgner-Kahrs are with the Continuum Robotics Laboratory, Department of Mathematical & Computational Sciences, University of Toronto, Mississauga, ON L5L 1C6, Canada (e-mail: priyankaprakash.rao@mail.utoronto.ca).

Oren Salzman is with the Department of Computer Science, Technion, Haifa 3200003, Israel (e-mail: osalzman@cs.technion.ac.il).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2024.3383211, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3383211

2377-3766 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

target pose, while avoiding contact with obstacles (Fig. 1(b)). To reach the target pose with obstacle avoidance, a multi segment robot is potentially required (Fig. 1(c)). A single segment long TDCR is essentially under-actuated for the task at hand as it only has two actuated DOFs (from bending and length insertion). However, CAN allows it to potentially leverage contacts with the obstacles to vary its curvature and reach the target pose, as shown in Fig. 1(d).

A. Related Work

Soft growing vine robots, despite being underactuated, leverage their follow-the-leader (FTL) deployment capability for CAN. However, the robot path is modeled as a series of line segments [7], [8] to simplify trajectory generation between obstacles, and does not reflect the TDCR's continuous backbone. Selvaggio et al. [6] use inverse kinematics to actively steering vine robots, but only look at point-sized obstacles. CAN has been exploited for hyperredundant robots where they have the advantage of additional DOFs that can be used to steer the end-effector towards the target pose. Sampling-based approaches like Rapidly-exploring Random Trees [9] can perform poorly when required to traverse through narrow passages present between obstacles [10]. Search-based planning in the configuration space has been proposed to reach a target position [11], a target pose [10] and to find stable load configurations [12], [13] for multi-link robots. While search-based planning offers a more systematic approach to exploring the workspace, the proposed heuristics in the above works are tailored for multi-link robots, which offer increased maneuverability (due to higher DOF) for controlling the end-effector's pose.

Therefore, CAN for a single segment TDCR necessitates methods that respect the robot kinematics and the fact that it is underactuated. Existing controllers account for possible contacts [14], [15], but require user-defined trajectories. Given a trajectory, inverse-kinematics models respecting contact mechanics [16] could be used to follow the trajectory as well. Naughton et al. [17] use reinforcement learning to develop the motion plan for a two-segment soft robot for position control amidst obstacles, although their work does not extend to achieving a target pose.

B. Contributions

In this letter, we propose a search-based motion planner for a planar single-segment TDCR to reach a target position and orientation. Our search-based planner discretizes the configuration space, and performs a best-first search while exploiting contacts with the environment. Key to the efficiency of our planner is a novel efficient-to-compute heuristic function. It captures the kinematics of the single segment TDCR and guides the planner to the goal, while exploring only a fraction of the entire search space.

II. Problem Definition

We consider a planar single-segment TDCR and denote its 2D workspace as $\mathcal{W} \in \mathbb{R}^2$, consisting of o known obstacles $\mathcal{W}_{\text{obs}} = \{\mathcal{O}_1, \dots \mathcal{O}_o\}$. The boundary of the jth obstacle is defined by a closed curve $f_j(\mathbf{x}) = 0$ with points lying outside the obstacle satisfying $f_j(\mathbf{x}) > 0$.

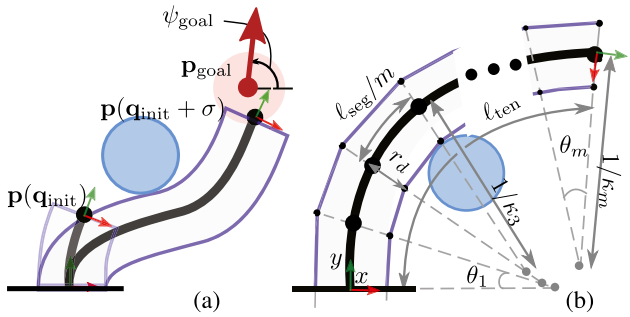


Fig. 2. (a) Robot shape at $\mathbf{q}_{\text{init}}$ and at $\mathbf{q}_{\text{init}} + \sigma$ with respective end-effector positions. (b) Workspace $\mathcal{W}$ with obstacles indicated in blue. The backbone is represented by a sequence of constant-curvature arcs subtending an angle θ_i with curvatures κ_i, where $i = 1, \dots, m$. The tendons are indicated in violet.

Our robot is actuated via (i) inserting and retracting the base of the robot and (ii) pulling and releasing a single tendon.[1] To this end, a *joint-space value*, $\mathbf{q} = (\ell_{\text{seg}}, \ell_{\text{ten}})$ describes the total length of the robot that has been inserted into the workspace and the tendon length, respectively. We denote the joint-space, namely the space of all joint-space values, as $\mathcal{Q}$. A *joint-space action* $\boldsymbol{\delta} = (\delta_{\text{seg}}, \delta_{\text{ten}})$ describes the amount δ_{seg} that the base is inserted or retracted (corresponding to positive and negative values of δ_{seg}, respectively) and the amount δ_{ten} that the tendon is releasaed or pulled (corresponding to positive and negative values of δ_{ten}, respectively). Applying a joint-space action $\boldsymbol{\delta} = (\delta_{\text{seg}}, \delta_{\text{ten}})$ on a robot with the joint-space value $\mathbf{q} = (\ell_{\text{seg}}, \ell_{\text{ten}})$ is denoted by $\mathbf{q} + \boldsymbol{\delta}$ and yields the new joint-space value $(\ell_{\text{seg}} + \delta_{\text{seg}}, \ell_{\text{ten}} + \delta_{\text{ten}})$. We will use σ to denote a *sequence* of joint-space actions and Σ to denote all such possible sequences.

Applying a joint-space action on a joint-space value is not always guaranteed to result in a feasible solution as the model might return a solution that does not satisfy the problem constraints. We refer to such actions as *invalid actions*, and all actions leading to feasible solutions as *valid actions*. Finally, we denote by $\mathbf{p}(\mathbf{q} + \sigma)$ and $\psi(\mathbf{q} + \sigma)$ the position and orientation of the robot's end-effector w.r.t. to the global frame of reference after applying σ to $\mathbf{q}$. Now, the motion planning problem can be stated as follows.

Problem 1: Given a single-segment TDCR with known shape known for an initial joint-space value $\mathbf{q}_{\text{init}} \in Q$, moving amidst a set $\mathcal{O} = \{\mathcal{O}_1, \dots \mathcal{O}_o\}$ of known obstacles, a tip goal pose, $\mathbf{T}_{\text{goal}}$ containing the goal position and orientation $\mathbf{p}_{\text{goal}}, \psi_{\text{goal}}$, a position and orientation tolerance ε, ω, our motion-planning problem calls for computing a sequence of valid joint space actions $\sigma = \{\boldsymbol{\delta}_1, \dots, \boldsymbol{\delta}_k\}$ such that

$$||\mathbf{p}(\mathbf{q}_{\text{init}} + \sigma) - \mathbf{p}_{\text{goal}}||_2 \leq \varepsilon, \tag{1}$$

$$|\psi(\mathbf{q}_{\text{init}} + \sigma) - \psi_{\text{goal}}| \leq \omega. \tag{2}$$

Namely, that the robot's tip reaches the goal pose within the goal tolerance as shown in Fig. 2(a). And that each joint-space action $\boldsymbol{\delta}_i$ is a valid action when applied to $\mathbf{q}_{\text{init}} + \boldsymbol{\delta}_1 + \dots \boldsymbol{\delta}_{i-1}$.

[1]The bending in a TDCR is underactuated, where even though there are two antagonistic tendons, and thus two inputs, the robot only has one degree of freedom in bending from tendon actuation. For simplification, we consider the length of one tendon, ℓ^1_{ten} as the actuation input. Once the corresponding robot configuration is calculated, the value of the second tendon, ℓ^2_{ten} can be determined using geometry. We remove the superscript of the first tendon for convenience.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

III. Kinematic Model for Obstacle Interaction

In this section we describe our kinematic model for a planar single-segment TDCR. Our approach adapts the model introduced by Ashwin et al. [18].[2] However, to reduce computational complexity, instead of employing the proposed four-bar mechanism, we use the so-called *piece-wise constant-curvature assumption* (PCCA). PCCA discretizes the robot into a series of m mutually-tangent constant-curvature arcs, each of curvature $\kappa_i, \forall i = 1, 2 \dots m$ as shown in Fig. 2(b). These individual curvatures are collectively represented by the vector $\boldsymbol{\kappa}$. The *configuration-space value* is represented by $\mathbf{c}(\mathbf{q}) = (\ell_{\text{seg}}, \boldsymbol{\kappa})$, and can be used to reconstruct the robot shape. As we will see, the kinematic model is treated as an energy-minimization problem, where the equilibrium configurations must (i) satisfy input tendon length constraints and (ii) remain outside the obstacles. In this section we first describe the optimization based kinematic model, and then use it to describe the required backbone parameters.

A. Optimization-Based Kinematic Model

The objective of the kinematic model is to determine the values of $\boldsymbol{\kappa}$ for a given joint-space value $\mathbf{q}$, while ensuring that the robot lies outside the obstacles. Since we consider only one tendon as the actuation input, the first nonlinear equality constraint enforces that the actuated tendon length, ℓ_{ten} should be equal to the calculated tendon length ℓ^*_{ten}. Calculating these tendon lengths for a configuration $\mathbf{c}$ is described in Section III-B.

$$g(\mathbf{c}(\mathbf{q})) = \ell^*_{\text{ten}} - \ell_{\text{ten}}. \tag{3}$$

Recall that the boundary of an obstacle j is defined by the function $f_j(\mathbf{x})$. Let angle $\psi_{f_j}(\mathbf{x})$ be the angle made by the tangent to this obstacle at $\mathbf{x}$. Since we want the robot to lie outside these obstacles, we can formulate the nonlinear constraint as

$$f_j(\mathbf{X}(\mathbf{c}(\mathbf{q}))) \geq 0, \tag{4}$$

where $\mathbf{X}(\mathbf{c}(\mathbf{q}))$ represents a set of points approximating the robot's geometry. The selection of these points is detailed in Section III-B. Assuming the material is isotropic, minimizing the strain energy at equilibrium configurations is equivalent to minimizing the individual curvatures of the subsegments [18]. The resulting optimisation problem, using (3) and (4), is given by

$$\begin{aligned} \min_{\boldsymbol{\kappa}} \quad & \sum_{i=1}^{n} (\kappa_i)^2 \\ \text{s.t.} \quad & g(\mathbf{c}(\mathbf{q})) = 0 \\ & f_j(\mathbf{X}(\mathbf{c}(\mathbf{q}))) \geq 0, \forall j = 1, 2 \dots o. \end{aligned} \tag{5}$$

B. Robot Kinematic Representation

Recall that we discretize the backbone into a series of m circular arcs, also called *subsegments*.[3] For a robot of length ℓ_{seg}, each sub-segment has a length of ℓ_{seg}/m and curvature of κ_i. Thus, the subtending angle of the i'th sub-segment is $\theta_i = \kappa_i \cdot \ell_{\text{seg}}/m$. The vector $\boldsymbol{\kappa}$ contains the curvatures of all these subsegments.

The tip of each subsegment is denoted by $\mathbf{p}_i$. If we define a local frame of reference attached to p_i with its z-axis tangent to the backbone, then the transformation matrix $\mathbf{T}^i_{i-1} \in \text{SE}(2)$ is used to calculate the relative pose between two such consecutive frames. We consider the two tendons placed at a distance of r_d from the backbone and denote their position vectors at the i'th sub-segment as $\mathbf{p}^1_{i,\text{ten}}$ and $\mathbf{p}^2_{i,\text{ten}}$ respectively.[4] Equation (3) requires calculating the length of the first tendon which is given by

$$\ell_{\text{ten}}(\mathbf{c}(\mathbf{q})) = \sum_{i=1}^{m} ||\mathbf{p}^1_{i,\text{ten}} - \mathbf{p}^1_{i-1,\text{ten}}||_2. \tag{6}$$

To define the nonlinear constraints in (4), we need to approximate the robot shape by a set of discrete points. In our setting, we will use a set of backbone and tendon positions. Specifically for a given $\mathbf{c}(\mathbf{q})$, we define the set $\mathbf{X}(\mathbf{q}, \boldsymbol{\kappa})$ of 3 m points as

$$\mathbf{X}(\mathbf{c}(\mathbf{q})) = \left\{ \mathbf{p}_i \quad \mathbf{p}^1_{i,\text{ten}} \quad \mathbf{p}^2_{i,\text{ten}}, \forall i = 1, \dots, m \right\}. \tag{7}$$

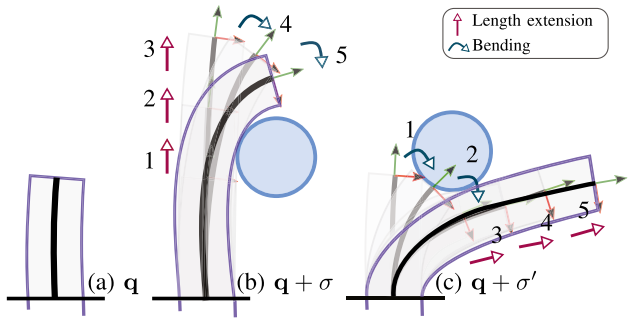


Fig. 3. (a) Robot's initial configuration at $\mathbf{q}$. (b), (c) Overlayed robot shapes for joint sequences (b) σ (two length-extensions, and three bending actions) and (c) σ' (reverse order: three bending actions, and two length-extensions).

C. Applying a Sequence of Joint Space Actions

In the absence of obstacles, there is a one-to-one mapping between a sequence of joint-space actions applied to the robot starting at some initial joint-space value and the final shape of the robot. However, contacts between the robot and the obstacles affect its shape. This implies that given an initial joint-space value $\mathbf{q} = (\ell_{\text{seg}}, \ell_{\text{ten}})$ and two different joint-space sequences σ, σ' such that $\mathbf{q} + \sigma = \mathbf{q} + \sigma'$, may result in different robot shapes due to different contact histories. An example is illustrated in Fig. 3. Therefore, there can be multiple solutions for (5).

To solve Problem 1, we begin with the initial joint-space value $\mathbf{q}_{\text{init}}$, with known $(\ell_{\text{init}}, \boldsymbol{\kappa}_{\text{init}})$. To compute the resulting shape after applying action $\boldsymbol{\delta}_1$ to $\mathbf{q}_{\text{init}}$, we use $\boldsymbol{\kappa}_{\text{init}}$ to warm-start the optimization in (5). The obtained curvature values can be used to calculate the end-effector position and orientation, and then used to solve for $(\mathbf{q}_{\text{init}} + \boldsymbol{\delta}_1 + \boldsymbol{\delta}_2)$. The model therefore, needs to be solved k times to obtain the $\boldsymbol{\kappa}$ for $(\mathbf{q}_{\text{init}} + \boldsymbol{\delta}_1 + \boldsymbol{\delta}_2 \dots + \boldsymbol{\delta}_k)$. Subsequently, the end-effector's position and orientation are determined.

[2]This model assumes there is no friction in the system and that the tendons are partially constrained [19].

[3]Such discretization is natural as typically, a TCDR is built using a series of m disks used to route the tendons.

[4]See [19] for the exact formulae.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

IV. Method

To address our motion-planning problem, we take a search-based planning approach [20], [21] where we run a best-first search on a discretized approximation of the continuous joint-space. This section details our implementation of the necessary primitive operations, data structures, and the proposed heuristic for this search.

A. Node Representation

Each node n in our search algorithm corresponds to a sequence of joint-space actions $\sigma(n) = \{\boldsymbol{\delta}_1, \ldots, \boldsymbol{\delta}_k\}$ and each edge corresponds to one such joint-space action δ_i. Each node is uniquely represented by pair $(\mathbf{q}^n, \boldsymbol{\kappa}^n)$ where $\mathbf{q}^n = \mathbf{q}_{\text{init}} + \delta_1 + \ldots + \delta_k$ is a joint-space vector and $\boldsymbol{\kappa}^n$ is a vector corresponding the individual curvatures of the robot's sub-segments. The curvatures $\boldsymbol{\kappa}^n$ resulting from applying $\sigma(n)$ to $\mathbf{q}_{\text{init}}$ can be obtained by solving the forward kinematics model defined in (5). The curvatures along with the segment length from the joint space vector can be used to reconstruct the shape of the robot using (7).

B. Node Extension

Our joint-space actions, $\mathcal{A} := (0, \delta\ell_{\text{ten}}), (\delta\ell_{\text{seg}}, \delta\ell_{\text{ten}})$, involve either adjusting the robot's tendon by $\delta\ell_{\text{ten}}$ or inserting its base by $\delta\ell_{\text{seg}}$ while simultaneously adjusting the tendon by $\delta\ell_{\text{ten}}$. For a node $n = (\mathbf{q}^n, \boldsymbol{\kappa}^n)$ and a joint-space action $\delta \in \mathcal{A}$, we define the operation of *extending* n by δ as creating a node $n' = (\mathbf{q}^{n'} = \mathbf{q}^n + \delta, \boldsymbol{\kappa}^{n'})$, where $\boldsymbol{\kappa}^{n'}$ is computed using the optimization in (5) with $\boldsymbol{\kappa}^n$ as the initial guess. We say that this extension is *valid* if δ is a valid joint-space action when applied to $\mathbf{q}^n$.

C. Duplicate Detection

To avoid expanding similar, yet non-identical states, existing search-based approaches over continuous search spaces (as is our setting) group states into equivalence classes [22]. This is analogous to how standard search algorithms (over finite discrete graphs) reduce search effort by identifying previously-visited states, a process known as *duplicate detection* [23]. In our case, two nodes $n = (\mathbf{q}^n, \boldsymbol{\kappa}^n)$ and $n' = (\mathbf{q}^{n'}, \boldsymbol{\kappa}^{n'})$ are considered duplicates if (i) $\mathbf{q}^n = \mathbf{q}^{n'}$ and (ii) $||\mathbf{p}(\sigma(n)) - \mathbf{p}(\sigma(n'))||_2 \leq d_{\text{sim}}$, where $d_{\text{sim}} > 0$ is some threshold parameter. That is, condition (ii) uses the distance between the two tip positions in order to detect duplicates.

D. Heuristic Estimate of Nodes

In heuristic-based search, heuristics are used to speed up the search by estimating the cost to reach the goal. In our case this means estimating the length of a path that will allow the end-effector of the TDCR to reach a target pose. In this letter, we propose a heuristic that considers the robot's limited steering properties around the known obstacles while avoiding the need to run the more computationally-complex forward kinematic model.

To achieve this, we suggest a simplified contact model for heuristic computation, which will employ constant-curvature (CC) arcs. Such arcs have been widely used to model load-free configurations [24]. Specifically, here we model the robot's motion via a sequence of paths having a constant curvature. The robot can only change its curvature along the backbone when in contact with an obstacle. Specifically, we assume that the shape, and consequently the end-effector's trajectory to the goal is a series of mutually tangent CC arcs, with each arc beginning and ending on the surface of an obstacle to reflect the approximated contact mechanics. Therefore, in the absence of any contacts this trajectory would be a single CC arc. While CC curves do not accurately reflect the robot shape as there is curvature variation due to contact forces, our work posits that they could be used to provide a sufficient estimate for the cost-to-go as they reflect the TDCR's limited steering capabilities.

Therefore, our heuristic $h(n) : \text{SE}(2) \to \mathbb{R}$ takes as input the position and orientation of the robot's end-effector at a given node n and estimates the length of the path to reach the goal. To avoid calculating $h(n)$ for every prospective node, we precompute $h(n)$ by performing a backward search from the goal. We discretize SE(2) into cells, each storing the shortest path to the goal calculated as a series of CC arcs, allowing us to quickly retrieve the heuristic value for any node during the search, detailed in IV-D4.

1) Overview: In order to precompute the cost of paths of CC arcs from each cell to the goal, we perform a backward search from the goal. We first find the set of all CC arcs whose end point orientation corresponds to the goal pose (Fig. 4(a)). From this set, we identify cells that could be potential contact points (Fig. 4(b)). From these contact cells, we perform the same search as before for CC arcs, but use the pose of these contact cells to populate consequent CC arcs (Fig. 4(c)). The process is repeated for all possible contact cells. Finally, we have a set of CC arcs from each cells that can reach the goal (Fig. 4(d)).

There are two main components to precomputing the heuristic: (1) `ComputeValidArcCells(pose)`: Determining the set of all cells from which a target pose is reachable with a *valid* CC arc (to be defined shortly), (2) `IdenitfyContact(cell)`: Identifying whether one of the cells determined in (1) is a possible contact cell, based on proximity to obstacles. This function is evaluated for each cell found in the above set. For the case where there are no obstacles, the heuristic computation only involves running step (1) with the goal pose as input.

2) Implementation: As mentioned, our heuristic will rely on a discretization of SE(2) into cells. Let this grid be denoted by $\mathtt{C}$, with each cell $c \in \mathtt{C}$ of size $\Delta \times \Delta \times \alpha$ where $\Delta \in \mathbb{R}$ and $\alpha = 2\pi/s$ for some $s \in \mathrm{N}^+$. Thus, we introduce mappings between cells and the pose they represent. Specifically, let `CELL` be a mapping taking a pose $(\mathbf{p}, \psi) \in \text{SE}(2)$ and returning the cell containing this pose. Similarly, let `POSITION`, `ORIENTATION` and `POSE` be mappings taking a cell $c \in \mathtt{C}$ and returning the position, orientation and pose at the center of c, respectively. The heuristic value of every cell is initially set to ∞.

(1)—`ComputeValidArcCells(pose)`: For any target pose $(\mathbf{p}, \psi) \in \text{SE}(2)$, there's a unique CC arc starting from $\mathbf{u} \in \mathbb{R}^2$ and reaching this target pose. Let $a = \mathtt{CC} - \mathtt{ARC}_{(\mathbf{p},\psi)}(\mathbf{u})$ be this arc. Its curvature, and length are denoted by $\kappa(a)$ and $\ell(a)$, respectively. Similarly, its orientation at $\mathbf{u}$ w.r.t. the global x-axis is denoted by $\psi(a)$. We say that such an arc is *valid* for some curvature bound $\kappa_{\max}$ if (i) $\kappa(a) < \kappa_{\max}$, (ii) $\theta(a) < \theta_{\max}$, and (iii) a does not intersect any obstacle.

For a position $\mathbf{u}$, let $C_{\mathbf{u}}$ be the set of all cells sharing the same position, but different in orientation. Given a target pose, if a *valid* arc a is computed, only one specific cell c_a in $C_{\mathbf{q}}$

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

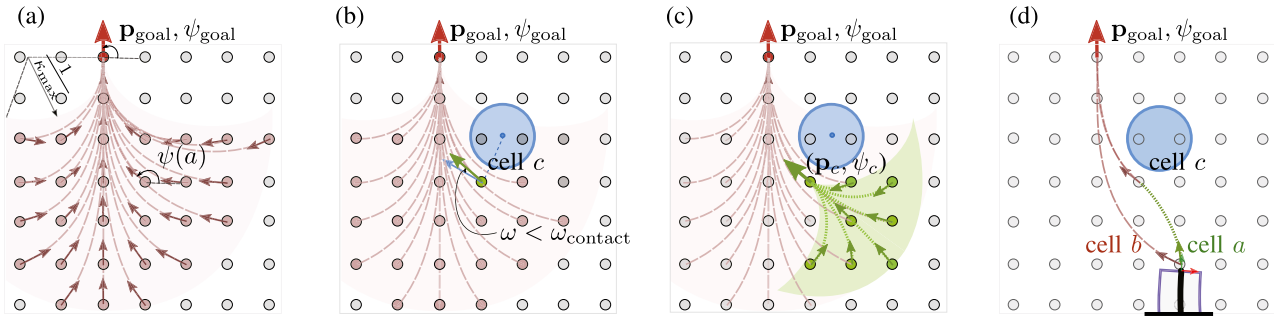


Fig. 4. Diagrammatic representation of the heuristic calculation. (a) `ComputeValidArcCells`$(\mathbf{p}_{\text{goal}}, \psi_{goal})$ evaluates cell poses that contain CC arcs (red) to reach a goal pose. Note that from each cell pose, only one arc reaches the goal. (b) Having obstacles (blue) invalidates some of the CC arcs. Running `IdenitfyContact(cell)` for each cell pose identified in the previous step identifies contact cells (green circles). (c) Using one of the contact cells, cell c, `ComputeValidArcCells`$(\mathbf{p}_c, \psi_c)$ is run, identifying new CC arcs that reach its pose (green) (d) After the above steps, we can see (i) positions from which arcs with several orientations were computed (e.g., cell b and cell a) and (ii) For the illustrated robot, the proposed heuristic informs the planner that based on its current end-effector orientation it can reach the goal via the CC arc connecting cell a and c, and consequently c to the goal.

Algorithm 1: Computing the Set of Cells That Can Reach a Given Pose via Valid CC Arcs.

Input: Target pose $(\mathbf{p}, \psi)$
Output: $S_{\mathbf{p},\psi}$
function HEURISTICARC(a)
  If $\kappa(a) > \kappa_{\max}$ or
$\theta(a) > \theta_{\max}$ or
    `InCollision`(a) **then**
    return ∞
  return $\ell(a)$
function COMPUTEVALIDARCCELLS($\mathbf{p}$, ψ)
  OPEN $\leftarrow(\mathbf{p})$
  $c \leftarrow$`CELL`$(\mathbf{p}, \psi)$
  $S_{(\mathbf{p},\psi)}$.insert(c)
  while not OPEN.empty() **do**
    $\mathbf{u} \leftarrow$OPEN.extract()
    $a \leftarrow$`CC` $-$ `ARC`$_{(\mathbf{p},\psi)}(\mathbf{u})$
    $c_a \leftarrow$`CELL`$(\mathbf{u}, \psi(a))$
    $h_a \leftarrow$HEURISTICARC(a)
    If $h_a < \infty$ **then**
      $h(c_a) \leftarrow \min\{h(c_a), h_a + h(c)\}$
      $S_{(\mathbf{p},\psi)}$.insert(c_a)
      for c' in `PositionNeighbor`(c_a) **do**
        OPEN.insert(`POSITION`(c'))
  return $S_{(\mathbf{p},\psi)}$

will match this arcs orientation $\psi(a)$, effectively linking the arc to a precise cell. The heuristic value h_a of cell c_a is set to the arc's length, $\ell(a)$, i.e. $h_a := \ell(a)$. We use a breadth-first search (outlined in Algorithm 1) to identify the set, $S_{(\mathbf{p},\psi)}$, of all cells with *valid* arcs reaching $(\mathbf{p}, \psi)$. This entire process is equivalent to identifying the workspace of a single CC segment as shown in Fig. 4(a).

(2)—`IdenitfyContact(cell)`: We say that a cell $c' \neq c$ is a *position neighbor* of c if their x and y coordinates are each within distance Δ of each other. A cell c is said to be a *contact cell* if (i) there exists a position neighbor c' of c whose position `POSITION`(c') lies within some obstacle $\mathcal{O}_j$ s.t. $\mathcal{O}_j \cap c \neq \emptyset$ and if (ii) the orientation of the cell differs from the local tangent of $\mathcal{O}_j$ by at most ω_{contact} i.e., $|$`ORIENTATION`$(c) - \psi_{f_j}($`POSITION`$(c'))| < \omega_{\text{contact}}$. Here, $\omega_{\text{contact}} > 0$ is some user-provided threshold.

3) Algorithm: To recall, we start by settling the initial heuristic value for all cells to ∞. We start by calculating the set of all cells $S_{(\mathbf{p}_{\text{goal}},\psi_{goal})}$ that have *valid* CC arcs directed towards the desired goal pose $(\mathbf{p}_{\text{goal}}, \psi_{goal})$. We do so by calling `ComputeValidArcCells`$(\mathbf{p}_{\text{goal}}, \psi_{goal})$ (detailed in Algorithm 1) that updates the heuristic value for each cell to be the length of the calculated arc.

We then repeat the following process (summarized in Algorithm 2) : For each c in the priority queue OPEN (extracted in a first-in first-out manner) where c is a contact cell and $h(c) < \infty$, we repeat the previous step by calling `ComputeValidArcCells`(`POSE`(c)) to update the heuristic value of each cell that can reach c. If c_a is one such cell that can reach c with a valid arc a, we set its heuristic value as $h(c_a) = \min\{h(c_a), h_a + h(c)\}$

4) Returning the Heuristic Value for a Node: Once the heuristic has been precomputed for grid cells using Algorithm 2, it can be used to return the heuristic value for a given node. Given a node n, we compute the end effector $(\mathbf{p}_n, \psi_n)$ of the robot at n. We then compute the cell c_n where this pose lies (i.e., $c_n =$ `CELL`$((\mathbf{p}_n, \psi_n))$. As described earlier, the CC assumption is only an approximation for the contact mechanics and does not precisely reflect contact mechanics. Thus, we introduce a final postprocessing smoothing step to account for orientation errors. Specifically, to set the heuristic value for c_n we also consider the values of it's neighboring cells c_n^+ and c_n^- which share the same position as c but differ in the orientation by a value of $\pm\alpha$. Namely,

$$c_n^+ := \texttt{CELL}(\mathbf{p}_n, \psi_n + \alpha) \quad \text{and}$$
$$c_n^- := \texttt{CELL}(\mathbf{p}_n, \psi_n - \alpha).$$

Our final heuristic value for the node n will be

$$h(n) = \min\{h(c_n), h(c_n^+), h(c_n^-)\}. \tag{8}$$

E. Putting It All Together—Search Algorithm

In this work, we implement a greedy best-first search algorithm [25]. Specifically, we make use of a priority queue OPEN that orders search nodes (Section IV-A) according to the the node's heuristic value (Section IV-D4).

The algorithm starts by initializing OPEN with a node corresponding to the initial configuration $\mathbf{q}_{\text{init}}$ and runs in iterations.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS FOR THE HEURISTIC CALCULATION AND MOTION PLANNER

Heuristic		Motion Planner	
Parameter	Value	Parameter	Value
Δ	0.001 m	ϵ	0.01 m
α	45°	ω	15°
$\kappa_{\max}$	250 m^{-1}	$\delta\ell_{\text{seg}}$	0.001 m
$\theta_{\max}$	270° m^{-1}	$\delta\ell_{\text{ten}}$	0.001 m
ω_{contact}	2.815°	$N_{\max}$	7000

Algorithm 2: Algorithm for Computing the Heuristic.

Input: Target pose $(\mathbf{p}_{\text{goal}}, \psi_{\text{goal}})$
Output: $h(c)\ \forall c \in \mathtt{C}$
function COMPUTEHEURISTIC($\mathbf{p}_{\text{goal}}, \psi_{\text{goal}}$)
  OPEN $\leftarrow (\mathbf{p}_{\text{goal}}, \psi_{\text{goal}})$
  $h(c) \leftarrow \infty\ \forall c \in \mathtt{C}$
  $h(\mathtt{CELL}((\mathbf{p}_{\text{goal}}, \psi_{goal})) \leftarrow 0$
  while not OPEN.empty()
    $(\mathbf{p}, \psi) \leftarrow$OPEN.extract()
    $S_{(\mathbf{p},\psi)} \leftarrow$COMPUTEVALIDARCCELLS($\mathbf{p}, \psi$)
    for c' in $S_{(\mathbf{p},\psi)}$ **do**
      if `IdentifyContact`(c') **then**
        OPEN.insert(`POSE`(c'))
  for c in $\mathtt{C}$ **do**
    $h(c) = \min\{h(c), h(c^+), h(c^-)\}$
  return h

Each iteration, the highest-priority node n is extracted from OPEN, and is *extended* (Section IV-B) by applying the set of actions $\mathcal{A}$ to its joint space values $\mathbf{q}^n$. For each successor node n', we compute its corresponding joint space values using our forward kinematic model (Section III). If n' is valid and not a duplicate (Section IV-C), it is added to OPEN. Finally, n is added to a so-called CLOSED set to avoid re-expanding previously expanded nodes.

This process continues until a node that satisfies the end-effector positional constraints specified in Problem 1, at which point the search is concluded successfully or until a maximal number of search iterations $N_{\max}$ is reached at which point the search is classified as unsuccessful.

V. EMPIRICAL EVALUATION

We evaluate our contact-based motion planner $\mathcal{M}_{\text{CAN}}$ with our heuristic in simulation.

1) Kinematic Model & Planner Parameters: The optimization problem in (5) is solved using an interior-point algorithm via MATLAB's *fmincon*. We consider a robot of radius 6 mm with a maximum length of 250 mm, represented by $m = 30$ arcs. The planner parameters are listed in Table I.

2) Environments: We consider three workspaces, $\mathcal{W}_1, \mathcal{W}_2$, and $\mathcal{W}_3$ (see Fig. 5). $\mathcal{W}_1$ has five equally sized convex circular obstacles while $\mathcal{W}_2$ features six circular obstacles with random centers and radii that overlap, creating non-convex shapes. Finally, $\mathcal{W}_3$ resembles a 2D cross-section of the interior of a turbine engine. CAN's potential use in such cluttered environments guides our future work (refer to Section VI). Specifically, the non-convex turbine blade shapes are approximated by four circular obstacles each. Recall that we compute a heuristic for the robot's end effector (Section IV-D) which can be modelled as a line segment translating and rotating in the plane. Thus, to ensure

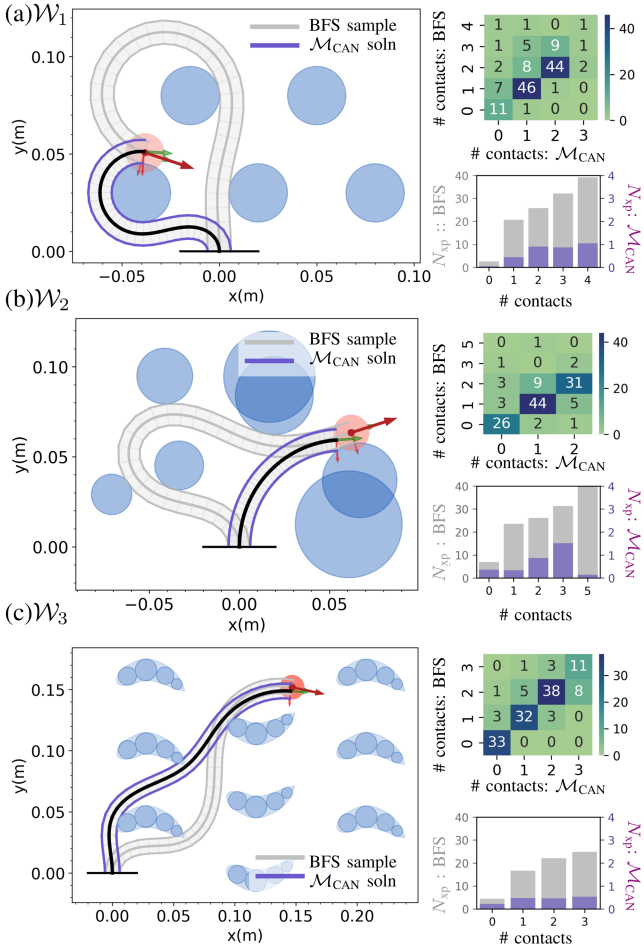


Fig. 5. Comparison of our motion planner $\mathcal{M}_{\text{CAN}}$ with the BFS planner used to generate the queries across three workspaces. For each plot we report (Top right) a heatmap showing the number of successful solutions for both BFS (y-axis) and $\mathcal{M}_{\text{CAN}}$ (x-axis), with numerical counts displayed within each cell. (Bottom right) N_{xp}, the average nodes expanded ($\times 1e3$ for $\mathcal{M}_{\text{CAN}}$ (purple) and BFS (grey)) with increasing number of contacts. Please note that the y-axis scale differs between the two planners.

that we only consider collision-free end effector placements, heuristic computation is done after inflating obstacles by an amount equal to the robot's width [26].

3) Queries: We generated for each workspace a set of valid queries as follows: We start by setting the initial configuration to $\mathbf{q}_{\text{init}} = (1, 1)$mm. We then run a breadth-first search (BFS) by varying the tendon and segment length by 1 mm. This creates a set of valid configurations with corresponding end-effector poses for which a solution to Problem 1 is guaranteed to exist. For each workspace, we sample 175 such end-effector poses at random.

4) Baselines: We compare our planner $\mathcal{M}_{\text{CAN}}$ with two baselines. The first baseline, which we denote by $\mathcal{M}_{\text{contactless}}$, is a planner that avoids contacts with the obstacles. Specifically, in the kinematic model, contactless configurations have constant curvatures. Thus, $\mathcal{M}_{\text{contactless}}$ performs inverse kinematics [24] for points satisfying the position tolerance and checks if the obtained solution satisfies the angular tolerance as well and does not intersect with obstacles. The second baseline, which we denote by $\mathcal{M}_{\text{CAN-simple}_h}$ is identical to our planner but uses a simple commonly-used heuristic function $h_{\text{simple}} := h_{xyz} +$

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUCCESS RATE [%] OF FINDING A SOLUTION FOR THE DIFFERENT PLANNING APPROACHES ON THE THREE WORKSPACES

	$\mathcal{W}_1$	$\mathcal{W}_2$	$\mathcal{W}_3$
$\mathcal{M}_{\text{CAN}}$ (proposed planner)	80.57	79.43	78.86
$\mathcal{M}_{\text{contactless}}$ (baseline 1)	16.57	25.14	26.86
$\mathcal{M}_{\text{CAN-simple}_h}$ (baseline 2)	29.71	19.42	28.57

$w \cdot h_\theta$ [27]. Here h_{xyz} represents the least-cost path from the tip to the goal, computed via Dijkstra's algorithm, h_θ is the absolute angular difference between the end-effector and the target orientation, and w is a constant used to balance the two, set to be 0.01.

5) Success Rate for Different Planners: In Table II, we present each planner's success rate in different workspaces. Both baselines show significantly lower success rates, underscoring the need for CAN and the proposed heuristic. Failures of $\mathcal{M}_{\text{CAN}}$ could be attributed to two factors: The first is that the heuristic's inexact approximation of contact mechanics may not always guide the robot successfully. The second is that, in many cases, if a particular path does not result for a given sequence of contacts, the planner tends to explore nearby nodes, without exploring paths with alternative contacts. A potential remedy is to discern similar configurations through soft duplicate detection [28], [29].

6) Comparing the Planner With BFS: We compared our planner $\mathcal{M}_{\text{CAN}}$ with the BFS planner used to generate queries, focusing on the number of contacts and the average nodes expanded (see Fig. 5). The heatmap analysis indicates that most solutions share a similar number of contacts, with a tendency for $\mathcal{M}_{\text{CAN}}$ to identify solutions with fewer contacts if possible. Notably, for $\mathcal{W}_2$ with five contacts, $\mathcal{M}_{\text{CAN}}$ finds a simpler solution, reflected in a lower value in the barplot. The barplot shows that the average number of nodes expanded by both planners increase with number of contacts, with BFS expanding more than 10 times the nodes expanded by $\mathcal{M}_{\text{CAN}}$.

7) Implementations on Motorized Prototype: Three sample motion plans in $\mathcal{W}_3$ were also implemented on a motorized prototype [30], and are shown in the supplementary video.[5] The executed plans on the prototype were compared with the simulations - the final robot shape had resulting tip errors of 4.4 mm, 7.8 mm, and 3.9 mm (3.66%, 4.38%, and 2.87% w.r.t length), which are comparable to model errors in literature. Notably, these errors were observed without any prior calibration of the robot's properties.

VI. CONCLUSION & FUTURE WORK

In this letter, we present a motion planner coupled with a novel heuristic that computes plans to reach a desired pose. Achieving a success rate of roughly 80% over 525 target poses amidst convex and non-convex obstacles. This efficiency stems from the proposed heuristic that enables the planner to examine just a fraction of the entire search space.

We consider several directions for future work which we briefly elaborate on:

1) Providing Guarantees on the Quality of the Solution: The heuristic we propose in Section IV-D is not guaranteed to be admissible, meaning it does not provide a lower bound on the cost-to-go. Consequently, there is not guarantee on the quality of the obtained solution by our planner. A potential solution to this limitation is employing multi-heuristic A* algorithms, which effectively incorporates both admissible and inadmissible heuristics in a structured manner [31], [32].

2) Extending the Planner to 3D Environments: To extend $\mathcal{M}_{\text{CAN}}$ for use in 3D environments, both the model and the heuristic require modifications. The design of the proposed motion planner is model-agnostic, allowing the potential use of any model in literature. The heuristic would require voxelizing the spaces $\mathbb{R}^3$ and $SO(3)$. Additionally, a challenge would be to represent the potential twists in the robot's structure due to contact interactions. To address this, the heuristic could be computed using arcs of uniform curvature along the x and y-axes, combined with a constant rate of twist around the z-axis.

An additional challenge is the computational effort: The current 2D kinematic model accounts for 99.3% of the computational effort, and shifting to a 3D model could increase this due to the complexity of TDCR modeling. Using a simplified approach, such as Euler curves [33], to model the backbone under contact forces might offer a feasible alternative. Another alternative may be to use multiple resolutions during the search [21], [34].

3) Using the Planner for Inspection of Turbine Engines: While 21 DOF hyperredundant robots have been investigated for turbine blade inspections [10], TDCRs with CAN offer the advantage of performing the same task with fewer DOFs. TDCRs mitigate the risk of blade damage due to their compliance. With our proposed motion planner targeting specific poses, future work will evolve into search-based inspection planning [35], focusing on surface inspection by orienting the robot appropriately.

ACKNOWLEDGMENT

The authors would like to thank Quentin Peyron for discussions aiding in the conceptualisation of this problem statement, Puspita Triana Dewi and Chloe Pogue for helping with prototyping and experiments.

REFERENCES

[1] M. Russo et al., "Continuum robots: An overview," *Adv. Intell. Syst.*, vol. 5, no. 5, 2023, Art. no. 2200367.
[2] A. Gao, H. Liu, Y. Zhou, Z. Yang, Z. Wang, and H. Li, "A cross-helical tendons actuated dexterous continuum manipulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2015, pp. 2012–2017.
[3] Y.-J. Kim, S. Cheng, S. Kim, and K. Iagnemma, "Design of a tubular snake-like manipulator with stiffening capability by layer jamming," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2012, pp. 4251–4256.
[4] C. Pogue, P. Rao, Q. Peyron, J. Kim, J. Burgner-Kahrs, and E. Diller, "Multiple curvatures in a tendon-driven continuum robot using a novel magnetic locking mechanism," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2022, pp. 472–479.
[5] M. B. Wooten and I. D. Walker, "Environmental interaction with continuum robots exploiting impact," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 10136–10143, Oct. 2022.
[6] M. Selvaggio, L. A. Ramirez, N. D. Naclerio, B. Siciliano, and E. W. Hawkes, "An obstacle-interaction planning method for navigation of actuated vine robots," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 3227–3233.
[7] J. D. Greer, L. H. Blumenschein, R. Alterovitz, E. W. Hawkes, and A. M. Okamura, "Robust navigation of a soft growing robot by exploiting contact with the environment," *Int. J. Robot. Res.*, vol. 39, no. 14, pp. 1724–1738, 2020.
[8] F. Fuentes and L. H. Blumenschein, "Mapping unknown environments through passive deformation of soft, growing robots," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2023, pp. 2522–2527.

[5]Can also be found at https://youtu.be/__tpUWCQx8A

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.

[9] L. Pfotzer, S. Klemm, A. Roennau, J. Zöllner, and R. Dillmann, "Autonomous navigation for reconfigurable snake-like robots in challenging, unknown environments," *Robot. Auton. Syst.*, vol. 89, pp. 123–135, 2017.
[10] M. S. Saleem, R. Sood, S. Onodera, R. Arora, H. Kanazawa, and M. Likhachev, "Search-based path planning for a high dimensional manipulator in cluttered environments using optimization-based primitives," in *Proc. IEEE Int. Conf. Intell. Robots Syst.*, 2021, pp. 8301–8308.
[11] A. Singh, Anshul C. Gong, and H. Choset, "Modelling and path planning of snake robot in cluttered environment," in *Proc. Int. Conf. Reconfigurable Mechanisms Robots*, 2018, pp. 1–6.
[12] X. Wang, M. Bilsky, and S. Bhattacharya, "Search-based configuration planning and motion control algorithms for a snake-like robot performing load-intensive operations," *Auton. Robots*, vol. 45, no. 8, pp. 1047–1076, 2021.
[13] R. Natarajan, G. L. Johnston, N. Simaan, M. Likhachev, and H. Choset, "Torque-limited manipulation planning through contact by interleaving graph search and trajectory optimization," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 8148–8154.
[14] M. C. Yip and D. B. Camarillo, "Model-less feedback control of continuum manipulators in constrained environments," *IEEE Trans. Robot.*, vol. 30, no. 4, pp. 880–889, Aug. 2014.
[15] Z. Zhang, J. Dequidt, J. Back, H. Liu, and C. Duriez, "Motion control of cable-driven continuum catheter robot through contacts," *IEEE Robot. Automat. Lett.*, vol. 4, no. 2, pp. 1852–1859, Apr. 2019.
[16] E. Coevoet, A. Escande, and C. Duriez, "Optimization-based inverse model of soft robots with contact handling," *IEEE Robot. Automat. Lett.*, vol. 2, no. 3, pp. 1413–1419, Jul. 2017.
[17] N. Naughton, J. Sun, A. Tekinalp, T. Parthasarathy, G. Chowdhary, and M. Gazzola, "Elastica: A compliant mechanics environment for soft robotic control," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 3389–3396, Apr. 2021.
[18] K. P. Ashwin, S. Kanti, and A. Ghosal, "Profile and contact force estimation of cable-driven continuum robots in presence of obstacles," *Mechanism Mach. Theory*, vol. 164, 2021, Art. no. 104404.
[19] P. Rao, Q. Peyron, S. Lilge, and J. Burgner-Kahrs, "How to model tendon-driven continuum robots and benchmark modelling performance," *Front. Robot. AI*, vol. 7, 2021, Art. no. 223.
[20] B. J. Cohen, M. Phillips, and M. Likhachev, "Planning single-arm manipulations with n-arm robots," in *Proc. Int. Symp. Combinatorial Search*, vol. 6, no. 1, pp. 226–227, 2015.
[21] M. Fu, O. Salzman, and R. Alterovitz, "Toward certifiable motion planning for medical steerable needles," in *Proc. Robot.: Sci. Syst.*, 2021.
[22] J. Barraquand and J.-C. Latombe, "Nonholonomic multibody mobile robots: Controllability and motion planning in the presence of obstacles," *Algorithmica*, vol. 10, pp. 122–155, 1993.
[23] P. A. Dow and R. E. Korf, "Duplicate avoidance in depth-first search with applications to treewidth," in *Proc. Int. Joint Conf. Artif. Intell.*, 2009, pp. 480–485.
[24] R. J. Webster and B. A. Jones, "Design and kinematic modeling of constant curvature continuum robots: A review," *Int. J. Robot. Res.*, vol. 29, no. 13, pp. 1661–1683, 2010.
[25] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. London, U.K.: Pearson, 2010.
[26] D. Halperin, O. Salzman, and M. Sharir, "Algorithmic motion planning," in *Handbook Discrete and Computational Geometry*. London, U.K.: Chapman and Hall/CRC, 2017, pp. 1311–1342.
[27] B. J. Cohen, S. Chitta, and M. Likhachev, "Search-based planning for manipulation with motion primitives," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2010, pp. 2902–2908.
[28] W. Du, S. Kim, O. Salzman, and M. Likhachev, "Escaping local minima in search-based planning using soft duplicate detection," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2019, pp. 2365–2371.
[29] N. Maray, A. Vemula, and M. Likhachev, "Improved soft duplicate detection in search-based motion planning," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 5792–5798.
[30] R. M. Grassmann, C. Shentu, T. Hamoda, P. T. Dewi, and J. Burgner-Kahrs, "Open continuum robotics–One actuation module to create them all," *Front. Robot. AI*, vol. 11, 2024, Art. no. 1272403.
[31] S. Aine, S. Swaminathan, V. Narayanan, V. Hwang, and M. Likhachev, "Multi-heuristic A," *Int. J. Robot. Res.*, vol. 35, no. 1/3, pp. 224–243, 2016.
[32] F. Islam, O. Salzman, and M. Likhachev, "Online, interactive user guidance for high-dimensional, constrained motion planning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 4921–4928.
[33] P. Rao, Q. Peyron, and J. Burgner-Kahrs, "Shape representation and modeling of tendon-driven continuum robots using Euler Arc Splines," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 8114–8121, Jul. 2022.
[34] M. Fu, K. Solovey, O. Salzman, and R. Alterovitz, "Resolution-optimal motion planning for steerable needles," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 9652–9659.
[35] M. Fu, A. Kuntz, O. Salzman, and R. Alterovitz, "Asymptotically optimal inspection planning via efficient near-optimal search on sampled roadmaps," *Int. J. Robot. Res.*, vol. 42, no. 4/5, pp. 150–175, 2023.

Authorized licensed use limited to: Shanghai Jiaotong University. Downloaded on April 29,2026 at 05:17:16 UTC from IEEE Xplore. Restrictions apply.